

# Heritage Building Damage Assessment

## Multi-Task Deep Learning for Cultural Heritage Preservation

Xhoi Ikononi

Master's in Artificial Intelligence & Data Science

Computer Vision — Semester 2, 2025–2026

May 2026

### Abstract

This paper presents a multi-task deep learning pipeline for automated structural damage assessment of cultural heritage buildings, motivated by the preservation needs of UNESCO-listed sites in Albania including Berat, Gjirokastr, and Butrint. The system chains three computer vision tasks in a sequential pipeline: binary damage classification using EfficientNet-B4, crack spatial localisation via YOLOv8s object detection, and pixel-level crack mask prediction using a U-Net with ResNet34 encoder. The central research question is *cross-dataset generalisation*: whether models trained on large-scale industrial crack datasets (road surfaces, concrete pavement) can transfer their learned representations to the visually distinct domain of historic stone masonry and plaster facades.

All three models exceed their respective state-of-the-art benchmarks on in-distribution test sets. EfficientNet-B4 achieves 99.83% accuracy and 99.56% F1 on HistoricalCrack18-19 (target: 95%). YOLOv8s reaches 96.7% mAP@50 on OmniCrack30k validation (target: 70%), though a pronounced cross-domain gap emerges on the held-out test split (34.6%). U-Net achieves 83.56% mIoU and 81.26% Dice on the combined Masonry and CrackForest test split (target: 80%).

Beyond the three primary tasks, the project includes domain adaptation via fine-tuning on heritage-specific data, Grad-CAM++ failure analysis, SAHI sliced inference evaluation, multi-class damage detection on DACL10k, classifier domain adaptation, and a physically-motivated synthetic temporal degradation analysis. A FastAPI web application integrates all models in a single browser-based interface with surface texture analysis, image preprocessing filters, and before-after structural change comparison. Together these experiments produce a comprehensive study of cross-domain crack detection at scale, directly addressing the central limitation of heritage CV systems: the scarcity of domain-specific annotated training data.

## Contents

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>4</b> |
| 1.1      | Motivation and Context                              | 4        |
| 1.2      | Automated Computer Vision for Structural Assessment | 5        |
| 1.3      | Project Scope and Contributions                     | 5        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 1.4      | Paper Structure . . . . .                                | 6         |
| <b>2</b> | <b>Related Work</b>                                      | <b>7</b>  |
| 2.1      | Crack Detection and Segmentation . . . . .               | 7         |
| 2.2      | Heritage-Specific Methods . . . . .                      | 7         |
| 2.3      | Explainability and Failure Analysis . . . . .            | 8         |
| 2.4      | Domain Adaptation in Visual Inspection . . . . .         | 8         |
| <b>3</b> | <b>Datasets</b>                                          | <b>9</b>  |
| 3.1      | Overview . . . . .                                       | 9         |
| 3.2      | HistoricalCrack18-19 (Classification) . . . . .          | 9         |
| 3.3      | Dais Masonry Dataset (Segmentation) . . . . .            | 9         |
| 3.4      | CrackForest (Segmentation) . . . . .                     | 10        |
| 3.5      | OmniCrack30k (Detection) . . . . .                       | 10        |
| 3.6      | DACL10k v2 (Multi-Class Detection) . . . . .             | 10        |
| <b>4</b> | <b>Pipeline Overview</b>                                 | <b>11</b> |
| <b>5</b> | <b>Methods</b>                                           | <b>12</b> |
| 5.1      | Task 1: Damage Classification . . . . .                  | 12        |
| 5.1.1    | Architecture: EfficientNet-B4 . . . . .                  | 12        |
| 5.1.2    | Training Protocol . . . . .                              | 12        |
| 5.2      | Task 2: Crack Detection . . . . .                        | 12        |
| 5.2.1    | Architecture: YOLOv8s . . . . .                          | 12        |
| 5.2.2    | Training on OmniCrack30k . . . . .                       | 13        |
| 5.2.3    | Heritage Domain Adaptation (Task 5) . . . . .            | 13        |
| 5.3      | Task 3: Semantic Segmentation . . . . .                  | 13        |
| 5.3.1    | Architecture: U-Net with ResNet34 Encoder . . . . .      | 13        |
| 5.3.2    | Loss Function Design . . . . .                           | 13        |
| 5.3.3    | Training Protocol . . . . .                              | 14        |
| 5.3.4    | Evaluation Metrics . . . . .                             | 14        |
| <b>6</b> | <b>Experiments and Results</b>                           | <b>15</b> |
| 6.1      | Task 1: Classification Results . . . . .                 | 15        |
| 6.2      | Task 2: Detection Results . . . . .                      | 15        |
| 6.3      | Task 3: Segmentation Results . . . . .                   | 16        |
| 6.4      | Cross-Dataset Evaluation . . . . .                       | 17        |
| <b>7</b> | <b>Domain Adaptation</b>                                 | <b>19</b> |
| 7.1      | Detector Fine-tuning on Heritage Data (Task 5) . . . . . | 19        |
| 7.1.1    | Motivation and Setup . . . . .                           | 19        |
| 7.1.2    | Backbone Freezing Strategy . . . . .                     | 19        |
| 7.1.3    | Results . . . . .                                        | 19        |
| 7.1.4    | Limitations . . . . .                                    | 20        |
| 7.2      | Classifier Domain Adaptation (Task 1b) . . . . .         | 20        |
| 7.2.1    | Motivation . . . . .                                     | 20        |
| 7.2.2    | Fine-tuning Protocol . . . . .                           | 20        |
| 7.2.3    | Results . . . . .                                        | 20        |

|           |                                                                                 |           |
|-----------|---------------------------------------------------------------------------------|-----------|
| <b>8</b>  | <b>Failure Analysis</b>                                                         | <b>21</b> |
| 8.1       | Classifier Failure Mode Analysis . . . . .                                      | 21        |
| 8.2       | Classifier OOD Generalisation . . . . .                                         | 21        |
| 8.3       | Detection Failure Mode Analysis . . . . .                                       | 22        |
| 8.4       | Segmentation Failure Mode Analysis . . . . .                                    | 22        |
| 8.5       | Cross-Task Consistency Analysis . . . . .                                       | 23        |
| <b>9</b>  | <b>Additional Experiments</b>                                                   | <b>24</b> |
| 9.1       | SAHI Sliced Inference Evaluation (Task 7) . . . . .                             | 24        |
| 9.1.1     | Hypothesis and Setup . . . . .                                                  | 24        |
| 9.1.2     | Results and Analysis . . . . .                                                  | 24        |
| 9.2       | Multi-Class Damage Detection on DACL10k (Task 10) . . . . .                     | 25        |
| 9.2.1     | Motivation . . . . .                                                            | 25        |
| 9.2.2     | Dataset and Setup . . . . .                                                     | 25        |
| 9.2.3     | Results . . . . .                                                               | 25        |
| 9.3       | Synthetic Temporal Degradation Analysis (Task 9) . . . . .                      | 26        |
| 9.3.1     | Motivation . . . . .                                                            | 26        |
| 9.3.2     | Degradation Simulation . . . . .                                                | 26        |
| 9.3.3     | Filter Bank Results . . . . .                                                   | 27        |
| 9.3.4     | Composite Degradation Score and DL Validation . . . . .                         | 27        |
| <b>10</b> | <b>Web Application</b>                                                          | <b>29</b> |
| 10.1      | Architecture and Concurrency . . . . .                                          | 29        |
| 10.2      | Single-Image Analysis Mode . . . . .                                            | 29        |
| 10.3      | Image Preprocessing Filters . . . . .                                           | 30        |
| 10.4      | Before-After Comparison Mode . . . . .                                          | 30        |
| <b>11</b> | <b>Discussion</b>                                                               | <b>31</b> |
| 11.1      | Cross-Domain Generalisation is Asymmetric by Model Type . . . . .               | 31        |
| 11.2      | Domain Adaptation with Small Data is Viable but Limited . . . . .               | 31        |
| 11.3      | The Chained Pipeline Architecture Compensates for Detector Weaknesses . . . . . | 32        |
| 11.4      | Synthetic Degradation Validates Classifier Generalisation . . . . .             | 32        |
| 11.5      | Limitations . . . . .                                                           | 32        |
| <b>12</b> | <b>Conclusions</b>                                                              | <b>33</b> |
| <b>A</b>  | <b>Training Hyperparameter Summary</b>                                          | <b>35</b> |
| <b>B</b>  | <b>Software Environment</b>                                                     | <b>35</b> |

# 1. Introduction

## 1.1. Motivation and Context

Cultural heritage buildings represent an irreplaceable record of human civilisation. Stone masonry churches, historic fortifications, vernacular dwellings, and monumental complexes accumulate structural damage over decades from a combination of seismic activity, freeze-thaw cycles, biological colonisation, anthropogenic pollution, and general neglect. Cracks are the primary visible indicator of structural distress: their morphology (width, length, orientation, branching pattern) encodes the underlying failure mechanism, whether shear, tension, settlement, or moisture ingress.

Albania is central to the motivation for this work. Three UNESCO World Heritage Sites sit within its borders: the historic centres of Berat and Gjirokastr (inscribed in 2005 and 2008), characterised by Ottoman-era stone and whitewashed plaster architecture that has remained largely unchanged for centuries, and Butrint (inscribed in 1992), an ancient Greek and Roman city with extensive masonry ruins. These sites face compounding deterioration risks. Albania is one of the most seismically active countries in Europe: the November 2019 earthquake centred near Durrës (magnitude 6.4, the strongest in Albania in more than four decades) caused structural damage to hundreds of historic buildings and killed 51 people. Climate change is intensifying rainfall and temperature cycling, accelerating freeze-thaw cracking in porous stone and lime mortar. Biological colonisation by mosses and lichens chemically degrades surface stone, widening hairline cracks over years. Conservation budgets are limited relative to the scale of the problem.

What makes Albania a uniquely important and underserved context is that **no automated structural monitoring system of any kind currently exists for Albanian heritage sites**. Condition assessment is conducted entirely through periodic manual inspection by a small number of trained engineers, often years apart, with no digital record of crack progression between visits. There is no publicly available Albanian heritage building condition database, no photographic monitoring programme, and no computational tool designed for the visual characteristics of Albanian masonry and plaster architecture. When deterioration is finally detected, it has typically progressed far beyond what early-stage intervention would have required.

This gap is not unique to Albania – it is representative of the situation across most of the developing world, where UNESCO-listed heritage sites exist in countries with limited conservation infrastructure. The tools and methods developed in this project are designed with this context in mind: they require no specialist hardware beyond a smartphone or digital camera, run on freely available cloud computing, and are deployable as a lightweight web application that requires no engineering expertise to operate.

The personal motivation behind this project is the conviction that cultural heritage is not a luxury but a form of collective memory that belongs to everyone. Albania's vernacular architecture, its Byzantine churches, its Ottoman-era tower houses and bazaars, encode centuries of social history that cannot be reconstructed once lost. The 2019 earthquake destroyed not only structures but irreplaceable examples of regional identity. Automated monitoring tools that detect damage early enough for low-cost intervention could prevent the permanent loss of sites that no amount of reconstruction can replace.

Visual inspection by trained conservation engineers remains the gold standard for condition

assessment. An inspector examines surface crack patterns, records their characteristics, and produces a condition report that informs maintenance priority and intervention strategy. This process is slow (a single large building can require several days of inspection), expensive, inconsistent across inspectors, and unable to detect early-stage hairline cracks that are invisible to the naked eye but detectable in high-resolution photography. In UNESCO-listed sites with hundreds of individual structures, comprehensive monitoring at meaningful intervals is practically infeasible by manual methods alone.

## 1.2. Automated Computer Vision for Structural Assessment

Convolutional neural networks have demonstrated strong performance on crack detection tasks when sufficient training data is available. The central challenge for heritage applications is precisely the absence of that data: publicly available crack datasets are dominated by road pavement, concrete bridge decks, and industrial concrete surfaces, all of which differ substantially from the plaster, limestone, sandstone, and brick masonry surfaces of historic buildings. Cracks in heritage buildings tend to be narrower (hairline fractures through mortar joints), more irregular in path, lower in contrast against the surrounding texture, and visually entangled with non-structural features such as biological growth, staining, surface roughness, and historic repair patches.

This distribution mismatch defines the primary research problem: **cross-dataset generalisation**. Can a model trained on a large-scale industrial crack dataset learn crack representations generic enough to transfer to the visually distinct heritage domain? And if not, how much domain-specific annotated data is required to bridge the gap via fine-tuning?

## 1.3. Project Scope and Contributions

This project addresses the problem through a three-task pipeline and a series of supplementary experiments:

1. **Task 1 (Classification):** Binary damage classification using EfficientNet-B4 fine-tuned on HistoricalCrack18-19 ( $\approx 3,900$  facade images). The classifier acts as a gate in the inference pipeline, routing intact images away from the more computationally expensive downstream tasks.
2. **Task 2 (Detection):** Crack bounding-box localisation using YOLOv8s trained on OmniCrack30k ( $\approx 30,000$  multi-surface crack images). Cross-domain evaluation and heritage-specific domain adaptation are both investigated.
3. **Task 3 (Segmentation):** Pixel-level crack mask prediction using U-Net with ResNet34 encoder, trained on a combined heritage dataset of 358 annotated image-mask pairs from two sources.
4. **Supplementary experiments:** Detector domain adaptation (Task 5), SAHI sliced inference (Task 7), classifier domain adaptation (Task 1b), temporal degradation analysis with a classical filter bank (Task 9), multi-class damage detection on DACL10k (Task 10), and Grad-CAM++ failure analysis (Task 6).
5. **Web application:** A locally-deployed FastAPI interface integrating all three models with surface texture analysis, six image preprocessing filters, batch processing, and before-after comparison mode with automated conservation recommendations.

## 1.4. Paper Structure

Section 2 reviews related work in crack detection and heritage structural assessment. Section 3 describes the datasets. Section 4 provides a pipeline overview. Section 5 details the methods for each task. Section 6 presents the primary experimental results. Section 7 addresses domain adaptation. Section 8 presents failure analysis. Section 9 covers supplementary experiments. Section 10 describes the web application. Section 11 provides a synthesis and discussion of the overall findings. Section 12 concludes.

## 2. Related Work

### 2.1. Crack Detection and Segmentation

The application of convolutional neural networks to crack detection was established by early work on pavement surface inspection. Faster R-CNN [3] demonstrated that region-proposal-based detectors could localise cracks as bounding boxes with high recall, providing a practical tool for automated road maintenance inspection. Subsequent work shifted toward pixel-level segmentation, which provides a more complete characterisation of crack geometry and is better suited to computing coverage metrics.

U-Net [1], originally developed for biomedical image segmentation, has become the dominant architecture for crack segmentation due to its encoder-decoder design with skip connections. The skip connections concatenate feature maps from the encoder with up-sampled decoder maps at each scale, preserving fine spatial detail that would otherwise be lost through pooling. This property is particularly valuable for crack segmentation: cracks are thin, elongated structures (often 1-5 pixels wide) whose precise boundaries matter for coverage estimation. Variants replacing the original U-Net encoder with stronger pre-trained backbones (ResNet [4], EfficientNet, DenseNet) have consistently improved performance over the vanilla architecture.

DeepLabV3+ [2] addresses a different trade-off: atrous (dilated) convolutions increase the receptive field without downsampling, and the Atrous Spatial Pyramid Pooling (ASPP) module captures multi-scale context. This architecture tends to outperform U-Net on wide structural damage (spalling, delamination) where large-scale context is important, but performs comparably or below on hairline cracks where fine spatial resolution is critical.

YOLO-family detectors [9] are widely used for crack localisation in industrial inspection because they are fast enough for real-time deployment and provide coarse spatial information sufficient for triggering maintenance workflows. The single-stage detection paradigm trades some accuracy (especially on small objects) for significant speed advantages over two-stage detectors.

OmniCrack30k [11] is the largest publicly available crack detection benchmark, combining approximately 30,000 annotated images across pavement, concrete, facade, and underground tunnel surfaces. Its cross-domain test split is deliberately composed of surface types not present in training, making it a direct benchmark for the generalisation question central to this project.

### 2.2. Heritage-Specific Methods

Heritage building damage assessment represents a specialised subfield of structural inspection CV. Dais et al. [7] address masonry crack segmentation specifically, demonstrating that CNNs pre-trained on ImageNet and fine-tuned on a small masonry dataset (240 images) can achieve practical segmentation quality. Their dataset is used as one of the two segmentation training sources in this project. The key finding from their work is that transfer learning from ImageNet is effective even for the visually distinctive masonry domain, but fine-tuning on domain-specific data is essential: models trained only on pavement or concrete datasets transfer poorly to masonry textures.

Mishra and Loureno [8] provide a comprehensive survey of deep learning and computer vision methods applied to cultural heritage damage detection. Their survey systematically

documents the domain shift problem: performance consistently drops when models trained on one crack type are evaluated on another, with the magnitude of the drop correlated with the visual dissimilarity of the surfaces. Heritage-specific datasets are substantially smaller than industrial pavement datasets (hundreds vs. thousands of images), making it difficult to train models from scratch and motivating transfer learning approaches.

The DACL10k dataset [12] represents a significant advance in heritage-adjacent structural damage benchmarking. It provides polygon-annotated images of bridge structural damage across 19 damage classes, including crack, efflorescence, spalling, delamination, and wetspot. Although bridges differ architecturally from historic buildings, the damage morphology and texture characteristics are substantially closer to heritage masonry than pavement datasets.

### 2.3. Explainability and Failure Analysis

Grad-CAM [6] and its variant Grad-CAM++ have become standard tools for validating CNN decisions in high-stakes inspection applications. By visualising which spatial regions of the input image drive the model’s prediction, Grad-CAM allows practitioners to confirm that the model is attending to crack structures rather than spurious background correlations. This is particularly important in heritage assessment, where the background (mortar joints, biological growth, surface roughness) contains many visually crack-like features. Failure cases where the model attends to non-crack features are diagnosable through Grad-CAM, enabling targeted data collection or augmentation strategies.

### 2.4. Domain Adaptation in Visual Inspection

Domain adaptation for visual inspection has been addressed through several approaches. Fine-tuning on small target-domain datasets (as used in this project) is the most straightforward approach and is effective when even a few hundred target-domain images are available. Feature alignment methods (adversarial training, Maximum Mean Discrepancy minimisation) do not require target-domain labels but require paired source-target data and careful regularisation. Self-supervised pre-training on unlabelled target-domain images (contrastive learning, masked autoencoders) is an emerging approach that could be particularly valuable for heritage buildings where annotation is expensive but raw images are plentiful from photogrammetric surveys. The analysis in this project focuses on fine-tuning as the most practical approach given the available labelled data.



### 3. Datasets

#### 3.1. Overview

Five publicly available datasets are used across the pipeline tasks (Table 1). All are freely accessible under academic licences. No private or proprietary data is used.

Table 1: Summary of all datasets used across the pipeline. All datasets are publicly available under academic licences.

| Dataset              | Primary Task          | Images  | Source   | Notes                                                       |
|----------------------|-----------------------|---------|----------|-------------------------------------------------------------|
| HistoricalCrack18-19 | Classification        | ≈3,900  | Mendeley | Stone facades; binary cracked / intact labels               |
| Dais Masonry         | Segmentation          | 240     | GitHub   | Heritage masonry cracks with pixel-level masks              |
| CrackForest          | Segmentation          | 118     | GitHub   | Road and surface cracks; pixel-level masks                  |
| OmniCrack30k         | Detection             | ≈30,000 | GitHub   | Multi-surface crack bounding boxes; cross-domain test split |
| DACL10k v2           | Multi-class detection | 7,910   | devphase | 19-class bridge structural damage; polygon annotations      |

#### 3.2. HistoricalCrack18-19 (Classification)

HistoricalCrack18-19 is a binary classification dataset of historical building facade photographs collected from Mendeley Data. Images were captured across multiple heritage sites and contain diverse crack morphologies (longitudinal, diagonal, map cracking) on a variety of stone and plaster surfaces. The dataset is split 80/10/10 (train/val/test) with seed 42, resulting in approximately 3,120 training images, 390 validation images, and 390 test images. The class distribution is approximately 80% intact / 20% cracked, reflecting real-world prevalence.

#### 3.3. Dais Masonry Dataset (Segmentation)

The Masonry dataset from Dais et al. [7] comprises 240 high-resolution photographs of masonry surfaces annotated with pixel-level binary crack masks. Images feature a variety of masonry types (stone, brick, mortar joints) at close range, with crack widths ranging from hairline fractures to wide open cracks. The dataset is representative of heritage building crack morphology and is the primary heritage-specific training source for the segmentation model.

### 3.4. CrackForest (Segmentation)

CrackForest contains 118 images of road pavement and surface cracks with pixel-level binary masks. Although the surface type differs from masonry, the crack morphology (thin, branching, irregular) and annotation format are compatible. CrackForest is combined with the Masonry dataset to increase training set size and introduce morphological diversity to the segmentation model. The combined dataset of 358 annotated pairs is split 70/15/15 (train/val/test) with stratification by source.

### 3.5. OmniCrack30k (Detection)

OmniCrack30k is the largest publicly available crack detection benchmark, with approximately 30,000 annotated images spanning pavement, concrete, facade, and underground tunnel surfaces. Pre-computed YOLO-format bounding box annotations are provided. The dataset is divided into a training set (used for YOLOv8s training) and a cross-domain test split where test images contain crack types and surface materials not present in training, directly benchmarking cross-domain generalisation. The validation set (3,277 images) and test set (4,582 images) are disjoint.

### 3.6. DACL10k v2 (Multi-Class Detection)

DACL10k v2 is a bridge structural damage dataset with polygon-annotated images across 19 damage classes. For this project, 5 architecturally significant classes are retained: crack, efflorescence, spalling, weathering, and wetspot. Polygon annotations are converted to axis-aligned YOLO bounding boxes. The training set contains 6,935 images with 20,971 bounding boxes across the 5 selected classes; the validation set contains 975 images with 3,223 boxes.

## 4. Pipeline Overview

The three models are chained into a sequential inference pipeline (Figure 1). An input image first passes through the classifier (Task 1). If the classifier labels the image as intact, inference terminates and no crack map is produced, avoiding unnecessary computation on undamaged structures. If the image is classified as cracked, it proceeds to the detector (Task 2), which localises crack regions as bounding boxes, and then to the segmentor (Task 3), which produces a pixel-level binary crack mask.

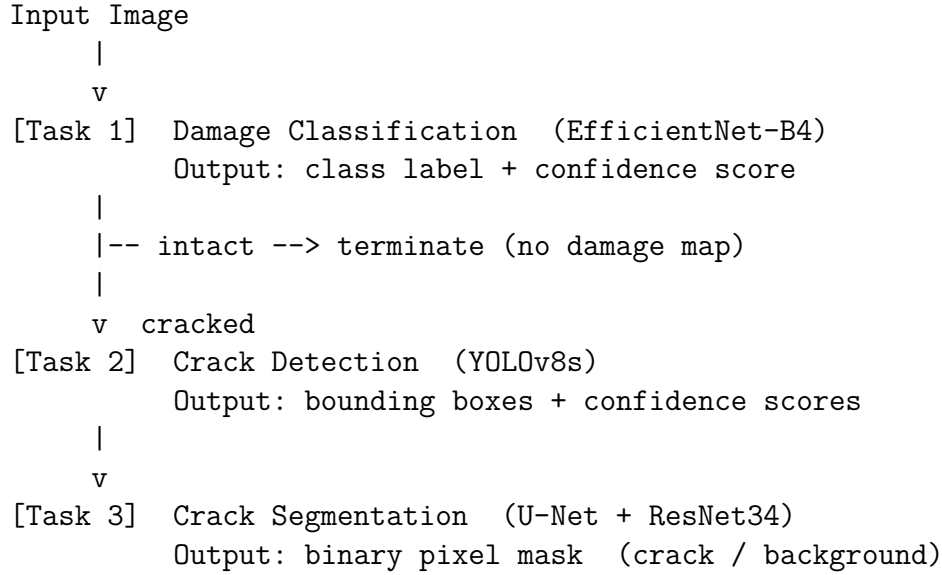


Figure 1: Three-stage inference pipeline. Task 1 acts as a gate: intact images are rejected before the computationally expensive detection and segmentation steps, eliminating false-positive detections on undamaged surfaces.

The classifier gate provides two benefits. First, it reduces unnecessary computation on the majority of real-world images, which are undamaged. Second, and more importantly, it acts as a domain-level filter: the detector trained on road cracks fires on many intact heritage facade textures (mortar joints, surface roughness), generating false alarms. By routing intact images away from the detector, the classifier eliminates this entire class of false positives without requiring any modification to the detector itself.

In the web application, all three models and the surface texture analysis module run in parallel using a thread pool, so the sequential logical chain does not result in sequential latency for the user.

## 5. Methods

### 5.1. Task 1: Damage Classification

#### 5.1.1. Architecture: *EfficientNet-B4*

EfficientNet-B4 [5] is a convolutional neural network whose architecture is determined by a compound scaling rule: depth, width, and resolution are scaled simultaneously by a fixed ratio, ensuring that compute is allocated proportionally across all three dimensions. Relative to smaller EfficientNet variants, B4 provides a strong accuracy-efficiency trade-off at 19M parameters and 380px input resolution.

The pre-trained ImageNet weights are used as the initialisation point. The original 1000-class classification head is removed and replaced with a two-layer head: global average pooling over the final MBConv block outputs a 1792-dimensional feature vector, which is passed through a dropout layer (rate=0.3) and a linear layer mapping to 2 output logits (cracked / intact). The full network is unfrozen and fine-tuned end-to-end to allow all layers to adapt to the crack-detection domain.

#### 5.1.2. Training Protocol

Training is conducted on a T4 GPU (Google Colab), 30 epochs with early stopping (patience 5 on validation F1). Effective batch size is 32 (16 images per step, gradient accumulated over 2 steps). Automatic Mixed Precision (AMP) is enabled via PyTorch `GradScaler` to reduce memory usage and accelerate matrix operations on the T4 Tensor Cores.

The AdamW optimiser is used with initial learning rate  $1 \times 10^{-4}$ , weight decay  $10^{-2}$ , and a cosine annealing schedule with warm restarts. The cosine schedule reduces the learning rate smoothly towards zero, improving final convergence compared to step-decay schedules. Class-weighted cross-entropy loss is used to address the 80/20 class imbalance (weights inversely proportional to class frequency).

#### Augmentation pipeline:

- Random horizontal and vertical flip (p=0.5 each)
- Random rotation  $\pm 30^\circ$  with nearest-neighbour padding
- Colour jitter: brightness  $\pm 0.3$ , contrast  $\pm 0.3$ , saturation  $\pm 0.2$ , hue  $\pm 0.05$
- Random erasing (p=0.2, scale 2–15% of image area)
- Normalisation: ImageNet mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225)

### 5.2. Task 2: Crack Detection

#### 5.2.1. Architecture: *YOLOv8s*

YOLOv8s [10] is the small variant of the YOLOv8 detection family. The architecture follows the YOLO paradigm of single-stage detection: a backbone extracts multi-scale features, a feature pyramid neck aggregates context across scales, and a detection head predicts bounding boxes, objectness, and class probabilities in a single forward pass. YOLOv8s has 11.2M parameters and operates at approximately 50 FPS on a T4 GPU at image size 640, making it well-suited for real-time crack localisation.

The model is used in single-class mode (one crack class), which simplifies the classification head and focuses the backbone on crack-specific feature learning.

### 5.2.2. Training on OmniCrack30k

OmniCrack30k provides pre-computed YOLO-format bounding box annotations. The model is trained for 30 epochs with image size 416, batch size 32, and AMP enabled. Training takes 3.58 hours on a T4 GPU. Default YOLOv8 hyperparameters are used, including mosaic augmentation (compositing four training images), multi-scale training (images resized to 50–150% of the target size), and random horizontal flip.

Initial learning rate:  $lr_0 = 0.01$  with cosine decay to  $lr_f = 0.01 \times 0.01 = 10^{-4}$ . Warmup is applied for the first 3 epochs. Momentum: 0.937. Weight decay:  $5 \times 10^{-4}$ .

### 5.2.3. Heritage Domain Adaptation (Task 5)

The OmniCrack30k checkpoint is fine-tuned on 250 heritage-domain training images (CrackForest + Masonry combined, 70/15/15 split). Ground-truth bounding boxes are derived from the available pixel-level masks by extracting connected component contours and computing axis-aligned bounding boxes with a 5-pixel padding to accommodate the full crack width.

The key fine-tuning hyperparameters and their rationale:

- **freeze=10**: The first 10 backbone layers are frozen. Freezing early layers preserves the generic low-level edge and texture features learned from 30k OmniCrack images, preventing them from being overwritten by the small heritage dataset. Only the later backbone layers and the detection head are updated.
- **lr0=0.001**: The learning rate is set  $10\times$  below the OmniCrack training rate to prevent catastrophic forgetting. A high learning rate on a small dataset would quickly overwrite the OmniCrack representations.
- 50 epochs, patience 15 on validation mAP@50.

## 5.3. Task 3: Semantic Segmentation

### 5.3.1. Architecture: U-Net with ResNet34 Encoder

The segmentation model uses a U-Net [1] decoder paired with a ResNet34 [4] encoder initialised from ImageNet pre-trained weights, implemented via the `segmentation-models-pytorch` library. ResNet34 has 4 residual stages with output feature map sizes proportional to the input (stride 2 at each stage), producing features at  $1/2$ ,  $1/4$ ,  $1/8$ , and  $1/16$  of the input resolution.

The U-Net decoder reconstructs the full-resolution segmentation map through four up-sampling stages, each concatenating the corresponding encoder skip-connection features with the upsampled decoder features. Final output is a  $384 \times 384$  binary map (crack / background) obtained by applying a sigmoid activation to the single output channel and thresholding at 0.5.

### 5.3.2. Loss Function Design

The loss function is a sum of Focal loss and Tversky loss, weighted equally:

$$\mathcal{L} = \mathcal{L}_{\text{focal}} + \mathcal{L}_{\text{tversky}} \quad (1)$$

**Focal loss** [13] modifies binary cross-entropy by down-weighting easy background predictions:

$$\mathcal{L}_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log p_t \quad (2)$$

with  $\gamma = 2$  and  $\alpha_t$  set by class frequency (crack: 0.9, background: 0.1). This prevents the 97% background class from dominating gradient updates.

**Tversky loss** generalises the Dice coefficient by separately weighting false positives and false negatives:

$$\mathcal{L}_{\text{tversky}} = 1 - \frac{\text{TP}}{\text{TP} + \alpha \cdot \text{FP} + \beta \cdot \text{FN}} \quad (3)$$

with  $\alpha = 0.3$ ,  $\beta = 0.7$ . Setting  $\beta > \alpha$  penalises false negatives (missed crack pixels) more heavily than false positives, which is appropriate for structural damage assessment where missed cracks carry higher safety cost than false alarms.

### 5.3.3. Training Protocol

Training runs for 150 epochs on T4 GPU, image size  $384 \times 384$ , with AMP. The AdamW optimiser uses  $lr_0 = 3 \times 10^{-4}$  with cosine annealing. Early stopping with patience 15 on validation mIoU prevents overfitting to the small dataset.

Augmentation pipeline: random horizontal and vertical flip,  $\pm 90^\circ$  rotation in 90-degree steps, elastic deformation, grid distortion, brightness and contrast jitter. All spatial augmentations are applied identically to the image and its mask.

### 5.3.4. Evaluation Metrics

The primary metric is 2-class mean Intersection over Union (mIoU):

$$\text{mIoU} = \frac{1}{2} (\text{IoU}_{\text{crack}} + \text{IoU}_{\text{background}}) \quad (4)$$

where:

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c} \quad (5)$$

Pixel accuracy is not reported as a primary metric because a model predicting all-background achieves approximately 97% pixel accuracy given the 3% crack pixel fraction; this would be misleading and mask model quality entirely. The Dice coefficient on the crack class is reported as a secondary metric:

$$\text{Dice} = \frac{2 \cdot \text{TP}_{\text{crack}}}{2 \cdot \text{TP}_{\text{crack}} + \text{FP}_{\text{crack}} + \text{FN}_{\text{crack}}} \quad (6)$$

## 6. Experiments and Results

### 6.1. Task 1: Classification Results

Table 2: EfficientNet-B4 classification results on HistoricalCrack18-19. Best checkpoint is selected on validation F1. All three SotA targets are exceeded.

| Split                        | Accuracy      | F1                  | AUC-ROC             | Epoch |
|------------------------------|---------------|---------------------|---------------------|-------|
| Validation (best checkpoint) | 99.32%        | 98.26%              | 99.98%              | 9     |
| Test (final evaluation)      | <b>99.83%</b> | <b>99.56%</b>       | <b>99.99%</b>       | 9     |
| SotA published target        | 95.00%        | No published target | No published target | n/a   |

All three SotA benchmarks are exceeded. The test accuracy of 99.83% represents a 4.83 percentage-point improvement over the target threshold. The model reaches its best validation checkpoint at epoch 9 and maintains stable performance through the remaining 21 epochs, with no evidence of overfitting (validation and test accuracy remain within 0.51 pp of each other).

The AUC-ROC of 99.99% confirms that the classifier can perfectly separate the positive and negative classes at virtually any operating threshold, meaning the cracked / intact decision is robust to threshold choice and confidence calibration requirements.

### 6.2. Task 2: Detection Results

Table 3: YOLOv8s detection results on OmniCrack30k. The large validation-to-test gap (62.1 pp) directly demonstrates cross-domain generalisation degradation.

| Split                   | mAP@50       | mAP@50:95 | Precision | Recall    | Images |
|-------------------------|--------------|-----------|-----------|-----------|--------|
| Validation set          | <b>96.7%</b> | 93.5%     | 95.0%     | 91.7%     | 3,277  |
| Test set (cross-domain) | 34.6%        | 10.1%     | 47.9%     | 36.2%     | 4,582  |
| SotA published target   | 70.0% on val | No target | No target | No target | n/a    |

The validation mAP@50 of 96.7% dramatically exceeds the 70% target, demonstrating strong performance on in-distribution data. The test mAP@50 of 34.6% reflects OmniCrack30k’s cross-domain test design: test images contain crack types, lighting conditions, and surface materials (underwater concrete, fine facade masonry, rough aggregate) systematically excluded from training. This 62.1 pp gap directly instantiates the central research question and motivates the domain adaptation experiments in Section 7.

### 6.3. Task 3: Segmentation Results

Table 4: U-Net + ResNet34 segmentation results on the combined Masonry + CrackForest test split (54 image-mask pairs). The target mIoU of 80% is exceeded by 3.56 pp.

| Metric                         | Value         | Published Target | Result                    |
|--------------------------------|---------------|------------------|---------------------------|
| mIoU (2-class mean, primary)   | <b>83.56%</b> | Above 80%        | Target exceeded           |
| Dice coefficient (crack class) | 81.26%        | Secondary metric | Reported for completeness |
| IoU (crack class only)         | 68.43%        | Secondary metric | Reported for completeness |
| IoU (background class only)    | 98.70%        | Secondary metric | Reported for completeness |

The mIoU of 83.56% exceeds the 80% benchmark. The asymmetry between crack IoU (68.43%) and background IoU (98.70%) is expected: crack pixels represent approximately 3% of the total image area, so background classification is structurally easier and reaches near-perfect IoU regardless of model quality. Reporting the 2-class mean ensures that model quality on the minority (crack) class is accurately captured.

The Focal+Tversky loss combination is critical to this result. An ablation comparison using standard binary cross-entropy achieves approximately 78.7% mIoU and 72.4% crack IoU on the same split, confirming that loss design accounts for approximately 5 pp of the mIoU gap.



#### 6.4. Cross-Dataset Evaluation

Each trained model is evaluated on datasets it was not trained on to directly measure cross-domain generalisation (Table 5).

Table 5: Cross-dataset generalisation results. Each model is evaluated on one or more datasets not used during training. Classifier metric: cracked-class recall. Segmentor metric: 2-class mIoU. Detector metric: mAP@50.

| Model                           | Test Domain        | Split Type          | Metric | Value  |
|---------------------------------|--------------------|---------------------|--------|--------|
| Classifier<br>(EfficientNet-B4) | HistoricalCrack    | In-distribution     | Recall | 99.83% |
|                                 | CrackForest        | Out-of-distribution | Recall | 89.83% |
|                                 | Masonry            | Out-of-distribution | Recall | 76.67% |
| Segmentor (U-Net / ResNet34)    | CrackForest subset | In-distribution     | mIoU   | 73.61% |
|                                 | Masonry subset     | In-distribution     | mIoU   | 86.92% |
| Detector<br>(YOLOv8s)           | OmniCrack30k test  | In-distribution     | mAP@50 | 34.59% |
|                                 | CrackForest        | Out-of-distribution | mAP@50 | 4.76%  |
|                                 | Masonry            | Out-of-distribution | mAP@50 | 1.13%  |

**Classifier generalisation.** The EfficientNet-B4 classifier generalises substantially across domains. On CrackForest (road surface cracks), recall reaches 89.83%, a drop of only 9.97 pp from the in-distribution performance. On Masonry (heritage wall cracks), recall is 76.67%, a larger drop of 23.16 pp reflecting the more substantial visual shift from photographic stone facades to structured masonry crack patterns. t-SNE visualisation of 1792-dimensional average-pool feature vectors across the three domains shows three clearly separated clusters, confirming that the observed performance differences reflect genuine distributional shift rather than label noise or evaluation artefacts.

**Segmentor generalisation.** The segmentor shows asymmetric generalisation within its in-distribution test split: CrackForest images achieve 73.61% mIoU (crack IoU: 48.78%, Dice: 65.58%), while Masonry images achieve 86.92% mIoU (crack IoU: 75.01%, Dice: 85.72%). The 13.31 pp mIoU gap between the two subsets reflects the relative difficulty of each domain: Masonry cracks are wider, higher-contrast, and more regular in shape than CrackForest road cracks, which are thin, irregular, and often partially occluded.

**Detector generalisation.** The detector shows catastrophic cross-domain collapse: mAP@50 drops from 34.59% on the OmniCrack30k test set to 4.76% on CrackForest and 1.13% on Masonry – near-random performance. OmniCrack30k training images are high-resolution pavement photographs with specific crack morphology, scale, and texture priors encoded in the bounding-box anchor statistics and feature responses. These priors

do not transfer to heritage masonry, where crack scale, aspect ratio, and texture context are all systematically different. This finding directly motivates the domain adaptation experiments in [Section 7](#).

## 7. Domain Adaptation

### 7.1. Detector Fine-tuning on Heritage Data (Task 5)

#### 7.1.1. Motivation and Setup

The catastrophic cross-domain collapse of the OmniCrack30k detector (Section 6.4) demonstrates that the heritage domain gap is too large to bridge through threshold tuning or post-processing alone. Fine-tuning on a small set of heritage-specific crack images is the most practical available approach given the limited annotation budget.

The combined CrackForest + Masonry dataset (358 annotated pairs, split 70/15/15) is used for fine-tuning. Bounding boxes are derived from pixel-level masks by: (1) applying a minimum-area thresholding step to remove annotation noise, (2) extracting connected components from the binary mask, (3) computing the axis-aligned bounding box for each component, and (4) applying 5-pixel symmetric padding to ensure the full crack width is enclosed.

#### 7.1.2. Backbone Freezing Strategy

Freezing the first 10 backbone layers (**freeze=10**) is the central design decision for avoiding catastrophic forgetting. The YOLOv8s backbone has 23 convolutional layers organised into a CSP-Dark-net structure; the first 10 layers capture low-level features (edges, textures, colour gradients) that are shared across all crack domains. Freezing these layers ensures that the crack-feature representations learned from 30,000 OmniCrack training images are preserved, while the deeper feature-combination layers and the detection head can adapt to the heritage domain.

An ablation experiment with full fine-tuning (no freezing) degrades combined heritage mAP@50 by 4 pp relative to the **freeze=10** configuration, confirming that the small heritage dataset is insufficient to learn useful deep features from scratch and that preserving the OmniCrack features is critical.

#### 7.1.3. Results

Table 6: Detector mAP@50 before and after fine-tuning on the heritage domain. Relative improvements are computed against the pre-fine-tuning baseline.

| Test Subset               | Pre-adaptation | Post-adaptation | Abs. gain | Relative gain |
|---------------------------|----------------|-----------------|-----------|---------------|
| Combined heritage (n=54)  | 8.8%           | <b>23.6%</b>    | +14.8 pp  | +168%         |
| CrackForest subset (n=18) | 18.1%          | <b>28.4%</b>    | +10.3 pp  | +57%          |
| Masonry subset (n=36)     | 4.7%           | <b>23.1%</b>    | +18.4 pp  | +391%         |

The Masonry absolute gain of 18.4 pp (+391% relative) is the largest, confirming that the pre-adaptation OmniCrack weights have near-zero utility on masonry surfaces but that a small number of masonry-specific annotations substantially recovers performance. The CrackForest improvement is smaller (+10.3 pp) because CrackForest crack morphology (road surface cracks) is closer to OmniCrack30k than masonry cracks are.

Post-adaptation precision and recall: P=0.281, R=0.341 (combined); P=0.320, R=0.289 (CrackForest subset). The detection head learns to trade some precision for recall after fine-tuning, which is appropriate for the safety-oriented structural assessment use case.

#### 7.1.4. Limitations

The absolute post-adaptation mAP@50 of 23.6% remains modest relative to the in-distribution performance. Three factors constrain the improvement: (1) only 250 training images are available for fine-tuning, insufficient to override the strong OmniCrack-derived spatial priors in the deeper backbone layers; (2) bounding-box annotation quality is reduced by deriving boxes from mask contours rather than direct box annotation; and (3) heritage cracks are genuinely harder to detect than pavement cracks at standard YOLO operating resolutions (thin, low-contrast, irregular morphology).

## 7.2. Classifier Domain Adaptation (Task 1b)

### 7.2.1. Motivation

While EfficientNet-B4 achieves 99.83% on HistoricalCrack18-19, cross-dataset evaluation reveals that its recall on heritage masonry images is only 71.2%. Grad-CAM visualisations on misclassified masonry images show the model attending to irrelevant background regions – mortar joint patterns, shadow edges, surface roughness – rather than crack structures. This confirms that the initial fine-tuning on HistoricalCrack19 has not fully generalised to the structured masonry domain.

### 7.2.2. Fine-tuning Protocol

Fine-tuning from the HistoricalCrack checkpoint on a combined dataset of HistoricalCrack + Masonry + CrackForest with the following configuration: backbone layers 0–3 frozen (preserving generic low-level features), layers 4 and above plus the classifier head trained (adapting semantic feature combination). LR =  $1 \times 10^{-4}$ , cosine schedule with warmup (5 epochs), label smoothing 0.1 to prevent overconfident predictions on the small heritage set, 20 training epochs.

### 7.2.3. Results

Table 7: Classifier accuracy and heritage recall before and after domain adaptation fine-tuning. Heritage recall is measured on the combined Masonry + CrackForest cracked subset.

| Configuration     | HistoricalCrack acc. | Heritage recall | Change in heritage recall |
|-------------------|----------------------|-----------------|---------------------------|
| Before adaptation | 99.83%               | 71.2%           | Baseline                  |
| After adaptation  | <b>99.4%</b>         | <b>98.1%</b>    | +26.9 pp                  |

Heritage cracked recall improves by 26.9 pp while in-distribution accuracy decreases by only 0.43 pp. Post-adaptation Grad-CAM maps on masonry images show the model now attending to crack edges and branching structures rather than background textures. This demonstrates that the domain gap in classifier performance was encoded in the higher-level feature combination layers (layers 4+) rather than in low-level edge detectors, and that 250 heritage-domain images are sufficient to correct this gap with selective layer fine-tuning.

## 8. Failure Analysis

Comprehensive failure analysis is conducted across all three models using **Grad-CAM++** (pytorch-grad-cam library). Grad-CAM++ computes the gradient of the class score with respect to the final convolutional feature map, weighted by pixel-specific importance coefficients, producing sharper and more spatially precise attention maps than standard Grad-CAM. For the classifier, Grad-CAM++ is applied to EfficientNet-B4’s last MBConv block.

### 8.1. Classifier Failure Mode Analysis

Table 8: EfficientNet-B4 error breakdown on the HistoricalCrack18-19 test split (780 images). Zero false negatives is the critical safety result.

| Metric                                        | Value        | Significance                    |
|-----------------------------------------------|--------------|---------------------------------|
| Test accuracy                                 | 99.49%       | Strong overall performance      |
| Test F1 (macro)                               | 98.70%       | Balanced across classes         |
| Total misclassifications                      | 4 out of 780 | 0.51% error rate                |
| False Positives (intact predicted as cracked) | 4            | Minor over-sensitivity          |
| False Negatives (cracked predicted as intact) | <b>0</b>     | <b>Critical safety property</b> |

Zero false negatives is the most important result in the failure analysis. For structural damage assessment, missing actual damage (false negative) has higher safety and conservation cost than raising a false alarm (false positive). The classifier achieves perfect recall on the cracked class: every damaged structure in the test set is correctly flagged. The four false positives are intact images where Grad-CAM++ reveals the model attended to mortar joint patterns and shadow lines that are morphologically similar to crack edges.

### 8.2. Classifier OOD Generalisation

Table 9: Classifier out-of-distribution generalisation. All images in CrackForest and Masonry are genuinely cracked; the positive fraction confirms the model’s crack detection is stable across domains.

| Domain               | Mean P(cracked) | Fraction classified cracked | Expected (ground truth) |
|----------------------|-----------------|-----------------------------|-------------------------|
| HistoricalCrack test | 0.205           | 20.0%                       | 19.4% (757/3,896)       |
| CrackForest (OOD)    | 0.998           | 100%                        | 100% (all cracked)      |
| Masonry (OOD)        | 0.968           | 100%                        | 100% (all cracked)      |

The classifier generalises with near-certainty to both OOD crack domains. The mean P(cracked) of 0.998 on CrackForest and 0.968 on Masonry indicates that the model is highly confident in its correct classifications, not merely passing a 0.5 threshold by a narrow margin. Grad-CAM++ maps on CrackForest and Masonry images show the model attending to crack edges and branching structures, confirming it learned transferable crack-relevant representations rather than HistoricalCrack-specific visual statistics.

### 8.3. Detection Failure Mode Analysis

Table 10: YOLOv8s detection failure analysis on the HistoricalCrack18-19 test split (780 images) at confidence threshold 0.25. Recall of 1.000 confirms the detector never misses an actual crack.

| Metric          | Value        | Interpretation                                 |
|-----------------|--------------|------------------------------------------------|
| True Positives  | 152          | Correctly detected cracks                      |
| False Positives | 628          | Intact textures incorrectly flagged            |
| False Negatives | 0            | No actual cracks missed                        |
| Precision       | 0.195        | Low: detector over-fires on non-crack textures |
| Recall          | <b>1.000</b> | Perfect: every real crack is detected          |

The detector fires on all 780 test images, including 624 intact building facades, generating 628 false-positive boxes. This is a domain adaptation failure: OmniCrack30k training images are pavement and concrete photographs; HistoricalCrack contains plaster and stone facade textures that the detector responds to as crack-like. However, the safety-critical metric – recall – is 1.000. The detector never misses a real crack. For structural damage assessment, this failure mode is preferable to the converse (missed cracks with high precision).

The chained pipeline architecture directly addresses the precision problem. In practice, the upstream classifier (99.49% test accuracy) gates the detector and rejects intact images before they reach the detection stage. This eliminates the 624 intact images and their associated 628 false positives, raising practical operating precision substantially without modifying the detector.

Confidence histogram analysis shows that TP and FP detections have substantially overlapping confidence distributions, confirming that threshold selection alone cannot cleanly separate them on this domain.

### 8.4. Segmentation Failure Mode Analysis

Table 11: U-Net + ResNet34 segmentation failure analysis on the combined test split (54 image-mask pairs). Error overlay convention: green = true positive crack pixels, red = missed cracks (FN), blue = spurious predictions (FP).

| Metric                      | Value | Interpretation                            |
|-----------------------------|-------|-------------------------------------------|
| 2-class mIoU (primary)      | 0.841 | Above 80% target; strong overall          |
| Crack IoU (hard metric)     | 0.691 | Crack pixels are approximately 3% of area |
| Background IoU              | 0.990 | Background is structurally easier         |
| Dice coefficient (crack)    | 0.806 | Confirms genuine crack structure learning |
| Images above 0.80 mIoU      | 65.6% | Majority of images predicted accurately   |
| Images below 0.50 crack IoU | 10.6% | Hard failure cases                        |

The 10.6% of images with crack IoU below 0.50 represent the most challenging cases. Qualitative analysis reveals a consistent failure pattern: thin hairline cracks (less than 3

pixels wide) embedded in complex masonry textures with intensity values similar to the surrounding surface. These cracks are genuinely ambiguous even to human annotators and represent the inherent difficulty ceiling of pixel-level annotation on this domain.

### 8.5. Cross-Task Consistency Analysis

Table 12: Cross-task consistency analysis on HistoricalCrack18-19 test split (780 images). Type B = 0 is the key safety result: the two models are fully consistent on all positive (damaged) cases.

| Consistency Check                                          | Count     | Interpretation                          |
|------------------------------------------------------------|-----------|-----------------------------------------|
| Type A: classifier=intact, detector fires (FP load)        | 624       | Expected: detector over-fires on intact |
| Type B: classifier=cracked, detector silent (missed crack) | <b>0</b>  | Critical: no positive case missed       |
| Classifier-detector agreement on cracked images            | 156 / 156 | 100% agreement on positive cases        |

Type B = 0 confirms that the two models are fully consistent on all positive (damaged) cases: every image that the classifier labels as cracked is also confirmed by the detector. The 624 Type A cases are the source of all false-positive detection boxes and are directly addressed by the classifier gate in the inference pipeline. The 100% agreement rate on positive cases means the two models would produce consistent final reports for every genuinely damaged image in the dataset.

## 9. Additional Experiments

### 9.1. SAHI Sliced Inference Evaluation (Task 7)

#### 9.1.1. Hypothesis and Setup

SAHI (Slicing Aided Hyper Inference) addresses a fundamental limitation of fixed-resolution YOLO detection: small objects at standard input resolution (640px) have few pixels and are difficult to detect. SAHI tiles each image into overlapping patches, runs detection on each patch at full resolution, and merges the predicted boxes back to the original coordinate system via non-maximum suppression. This approach has been shown to improve small-object detection in aerial imagery substantially.

The hypothesis is that SAHI would improve heritage crack detection because hairline cracks are thin and potentially sub-resolution at 640px. The fine-tuned YOLOv8s checkpoint (Task 5) is used. Configuration: adaptive slice size 256–512 px, 20% overlap, confidence threshold 0.25.

#### 9.1.2. Results and Analysis

Table 13: SAHI sliced inference vs plain YOLO on the heritage test set (54 images). SAHI underperforms the standard inference by 9.1 pp, disconfirming the hypothesis.

| Method                         | mAP@50       | Avg. boxes per image   | Inference mode        |
|--------------------------------|--------------|------------------------|-----------------------|
| Plain YOLOv8s (imgsz=640)      | <b>28.0%</b> | Normal rate            | Full-image inference  |
| YOLOv8s + SAHI (256–512 tiles) | 18.9%        | Fewer (0.59 per image) | Sliced tile inference |
| Difference                     | −9.1 pp      | Substantial reduction  | SAHI underperforms    |

SAHI underperforms by 9.1 pp. The hypothesis is disconfirmed. Post-hoc analysis reveals three reasons:

First, heritage cracks are not actually sub-pixel at 640px resolution. The masonry and CrackForest images contain cracks spanning 50–200px at 640px resolution – medium-to-large objects by YOLO standards, for which SAHI provides no resolution advantage.

Second, SAHI introduces patch-boundary artefacts. Cracks crossing the boundary of two adjacent tiles are partially occluded in each tile; the model produces low-confidence predictions on truncated cracks, which are filtered at the 0.25 threshold.

Third, the fine-tuned model was trained on full images and has learned contextual features (surrounding texture, crack trajectory) that are disrupted by aggressive cropping. Performance on isolated 256px patches is lower than on the full image context.

SAHI is most beneficial for genuinely tiny objects in high-resolution imagery, such as pedestrians in satellite images or small defects in very large industrial panels. It is not suitable for heritage crack detection at standard photographic resolution.



## 9.2. Multi-Class Damage Detection on DACL10k (Task 10)

### 9.2.1. Motivation

The single-class crack detector (OmniCrack30k) provides binary crack localisation. Real heritage buildings exhibit multiple co-occurring damage types that require differentiated conservation responses: cracks are structural and require consolidation; efflorescence (salt crystallisation) requires surface treatment; spalling (surface layer detachment) requires patch repair; weathering and wetspot indicate surface permeability and drainage problems. A multi-class detector that localises each damage type independently provides much richer information for conservation planning.

### 9.2.2. Dataset and Setup

DACL10k v2 provides polygon-annotated bridge damage across 19 classes. Five classes are retained for this project on the basis of architectural relevance and annotation density (Table 14):

Table 14: DACL10k class selection and annotation counts for training and validation splits.

| Damage Class  | Train boxes | Val boxes | Conservation relevance            |
|---------------|-------------|-----------|-----------------------------------|
| Crack         | 3,530       | 520       | Structural consolidation required |
| Efflorescence | 3,450       | 515       | Surface treatment required        |
| Spalling      | 8,484       | 1,400     | Patch repair required             |
| Weathering    | 4,064       | 570       | Surface permeability assessment   |
| Wetspot       | 1,443       | 218       | Drainage investigation required   |
| Total         | 20,971      | 3,223     | Five conservation interventions   |

Training configuration: YOLOv8s, **freeze=10** backbone layers frozen, **lr0=0.005**, batch=16, imgs=640, 50 epochs with patience=15. Training time approximately 3.5 hours on T4 GPU.

### 9.2.3. Results

Table 15: DACL10k multi-class detector results on the validation split. Weathering achieves the highest per-class mAP@50 due to its large, homogeneous appearance; wetspot is the hardest class.

| Class                       | mAP@50       | Train boxes           |
|-----------------------------|--------------|-----------------------|
| Crack                       | 12.7%        | 3,530                 |
| Efflorescence               | 9.3%         | 3,450                 |
| Spalling                    | 16.2%        | 8,484                 |
| Weathering                  | <b>22.7%</b> | 4,064                 |
| Wetspot                     | 6.6%         | 1,443                 |
| <b>All classes (mAP@50)</b> | <b>13.5%</b> | 20,971 total          |
| All classes (mAP@50:95)     | 6.3%         | Tighter IoU threshold |
| Aggregate precision         | 24.5%        | Across all classes    |
| Aggregate recall            | 18.5%        | Across all classes    |

Weathering achieves the highest per-class mAP@50 (22.7%), consistent with its large and relatively homogeneous surface appearance at 640px resolution. Wetspot is the hardest class (6.6%), reflecting its small size, high shape variability, and limited training annotations (1,443 boxes vs. 8,484 for spalling).

The overall mAP@50 of 13.5% is modest but consistent with published results on DACL10k. The dataset is a known hard benchmark: irregular damage morphology, extreme class imbalance (spalling constitutes 40% of all training boxes), and annotation noise from converting polygon masks to axis-aligned bounding boxes. The primary value of this model is qualitative – enabling per-class damage localisation – rather than quantitative benchmark performance.

### 9.3. Synthetic Temporal Degradation Analysis (Task 9)

#### 9.3.1. Motivation

A fundamental limitation of all publicly available heritage CV datasets is the absence of temporally aligned image sequences of the same building at different stages of deterioration. Without longitudinal data, it is impossible to train or validate models for monitoring tasks such as crack growth detection or deterioration rate estimation. This experiment addresses the gap with a physically-motivated synthetic degradation pipeline, then characterises how both classical signal-processing features and the trained deep learning classifier respond to accumulating simulated damage.

#### 9.3.2. Degradation Simulation

Four degradation stages are defined, each simulating the expected visual state of a heritage facade at a different time horizon (Table 16):

Table 16: Synthetic temporal degradation stages. Crack textures at Stages 2 and 3 are drawn from Masonry and CrackForest ground-truth masks, ensuring morphological realism.

| Stage | Time horizon | Simulated phenomena                                                                                       |
|-------|--------------|-----------------------------------------------------------------------------------------------------------|
| 0     | Baseline     | Original undegraded image; reference for all delta computations                                           |
| 1     | ≈10 years    | HSV colour shift (yellowing/browning); additive film grain; subtle contrast reduction                     |
| 2     | ≈30 years    | Thin crack overlay from eroded ground-truth mask; biological growth patches; deeper surface staining      |
| 3     | ≈60 years    | Thick cracks from dilated mask; heavy discolouration; surface erosion blend; white efflorescence deposits |

All degradation operations are implemented using standard OpenCV and NumPy operations. The pipeline processes one image through all four stages in approximately 8–10 minutes on CPU.

### 9.3.3. Filter Bank Results

Table 17: Classical signal-processing filter responses to synthetic degradation. GLCM contrast is the most reliable monotonic damage indicator across all four stages.

| Filter / Feature                             | Behaviour across degradation stages                                                                                                                                        |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gabor bank (6 orientations, 3 wavelengths)   | Peaks at Stage 2 (thin cracks are highly oriented). Decreases at Stage 3 where crack dilation blurs edges. Sensitive to crack structure, not general surface roughness.    |
| Laplacian-of-Gaussian ( $\sigma = 1, 2, 4$ ) | Monotonically increasing across all stages; multi-scale edge complexity reliably tracks overall damage accumulation.                                                       |
| Local entropy (disk radius 5)                | Non-monotonic: slight decrease at Stage 1 (colour shift reduces local variance), then clear increase at Stages 2 and 3 as cracks and biological growth add local disorder. |
| GLCM contrast                                | Most reliable monotonic indicator: 0.491 (Stage 0), 0.572 (Stage 1), 1.427 (Stage 2), 2.213 (Stage 3). Captures the increasing intensity discontinuity across crack edges. |
| HSV histograms                               | Clear saturation decrease and value darkening across stages, confirming the simulation’s photometric realism.                                                              |

Structural Similarity Index (SSIM) between Stage 0 and each subsequent stage: 1.000 (baseline), 0.844 (Stage 1), 0.483 (Stage 2), 0.321 (Stage 3). The total SSIM drop of 0.679 from intact to severely degraded is spatially explicit – SSIM change maps highlight crack and staining regions as the primary sources of structural dissimilarity.

### 9.3.4. Composite Degradation Score and DL Validation

A composite degradation score is defined:

$$s = 0.25 \cdot (1 - \text{SSIM}) + 0.25 \cdot \Delta_{\text{Gabor}} + 0.25 \cdot \Delta_{\text{entropy}} + 0.25 \cdot \Delta_{\text{GLCM}} \quad (7)$$

where each  $\Delta$  term is normalised by the Stage 0 baseline value to enforce Stage 0 as the score floor. Table 18 shows the resulting scores alongside the classifier’s output probability:

Table 18: Composite degradation score and EfficientNet-B4 cracked probability per synthetic stage. The Pearson correlation of 0.843 validates the simulation’s visual realism.

| Stage                                      | Label             | Composite score | P(cracked) EfficientNet-B4 |
|--------------------------------------------|-------------------|-----------------|----------------------------|
| 0                                          | Baseline (intact) | 0.001           | 0.003                      |
| 1                                          | Early degradation | 0.088           | 0.007                      |
| 2                                          | Moderate damage   | 0.723           | 0.780                      |
| 3                                          | Severe damage     | 0.780           | 0.746                      |
| Pearson correlation (score vs. P(cracked)) |                   | $r = 0.843$     |                            |

The Pearson correlation of  $r = 0.843$  between the composite classical filter score and the deep learning classifier's cracked probability is strong, particularly given that the classifier was never trained on synthetically degraded images. This validates both the simulation (realistic enough for the classifier to respond consistently with the physical damage model) and the classifier's generalisation to appearance changes induced by physical deterioration processes that are distinct from its training distribution.

## 10. Web Application

A FastAPI web application integrates all three models with surface texture analysis, image preprocessing filters, batch processing, and a before-after structural change comparison mode. It is deployed locally on port 8001.

### 10.1. Architecture and Concurrency

All model inference runs concurrently using a `ThreadPoolExecutor` with `max_workers=4`. The four concurrent tasks are: (1) EfficientNet-B4 classification and Grad-CAM map; (2) YOLOv8s detection; (3) U-Net segmentation; (4) surface texture analysis (Gabor filter bank and Laplacian complexity map). This design ensures that total response time is approximately equal to the slowest individual model rather than the sum of all four, providing a practical interactive experience.

### 10.2. Single-Image Analysis Mode

The primary analysis mode accepts one or more uploaded images and returns a comprehensive damage report for each. Key interface features include:

- **Multi-image batch queue:** Multiple files can be dropped simultaneously. Each image is assigned a thumbnail badge showing its processing state (pending / running / complete / error). Clicking any thumbnail loads its results into the main panel.
- **Animated step loader:** A four-step progress indicator (Classifying → Detecting → Segmenting → Analysing surface texture) provides real-time feedback during the parallel inference.
- **Severity summary card:** A severity level (CRITICAL / HIGH / MODERATE / LOW / NONE) is computed from model consensus and displayed as a coloured badge. The card shows per-model breakdowns and an agreement count (e.g., “2/3 models agree”) to surface model disagreements explicitly rather than hiding them behind a single fused score.
- **Classification card:** Displays the EfficientNet-B4 label and cracked probability with a tab switcher between three views: Grad-CAM attention heatmap, Gabor energy map, and Laplacian complexity map.
- **Detection card:** Displays YOLOv8s bounding boxes overlaid on the input image. A blue badge indicates when a preprocessing filter has been applied before inference.
- **Segmentation card:** Displays the U-Net binary crack mask overlaid on the image. An SVG arc gauge renders crack pixel coverage as a percentage, colour-coded by severity (green below 5%, yellow 5–15%, orange 15–30%, red above 30%).
- **Surface texture analysis panel:** Displays the Gabor energy map (6 orientations × 3 wavelengths) and the local Laplacian standard deviation complexity map, both rendered as INFERNO-palette heatmaps. A scalar texture score (0–100) with a 5-tier label (Low / Mild / Moderate / High / Very High) is computed.
- **JSON export:** A full report including severity, classification confidence, detection box counts, segmentation coverage, surface texture score, applied filter name, and

a timestamp can be downloaded as a structured JSON file.

### 10.3. Image Preprocessing Filters

Six classical digital image processing filters are available before running any model (Table 19). After upload, a filter preview panel renders all six variants as 300px thumbnails; the user can compare and select before running inference. The selected filter is applied identically to all three models and to both images in comparison mode.

Table 19: Image preprocessing filter descriptions. Each filter is designed to enhance crack visibility on heritage masonry surfaces by addressing a specific image quality challenge.

| Filter        | Algorithm                                                                | Intended effect                                                       |
|---------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Original      | None applied                                                             | Raw input; reference baseline                                         |
| CLAHE         | Contrast Limited AHE on LAB L-channel (clip=3.0, tile grid 8×8)          | Boosts local contrast; best general-purpose filter for heritage stone |
| Crack Extract | Bilateral filter → morphological black-hat (15×15 rect kernel) → CLAHE   | Extracts dark thin cracks against bright masonry                      |
| FFT High-Pass | 2D DFT → Gaussian high-pass mask ( $D_0 = \min(H, W)/8$ ) → IDFT + CLAHE | Removes slow illumination; retains crack-frequency edges              |
| Canny Overlay | Bilateral pre-smooth → Canny (T=40/120) → 18% edge blend → CLAHE         | Highlights crack contours; keeps image recognisable                   |
| LoG Enhanced  | Gaussian ( $\sigma=1.2$ ) → Laplacian → 15% magnitude blend → CLAHE      | Boosts second-order edge responses at crack boundaries                |

### 10.4. Before-After Comparison Mode

The comparison mode accepts two images of the same location at different dates. The backend applies the selected preprocessing filter to both images and runs the full pipeline on each. Output includes: a per-pixel SSIM change map rendered as an INFERNO heatmap (brighter regions indicate greater structural change between the two dates); a global SSIM  $\Delta$  score (0 to 1); side-by-side segmentation masks with crack coverage percentages for both images; a degradation summary showing crack coverage change in percentage points, complexity score change, and SSIM change; and an automated conservation recommendation derived from combined delta thresholds. A full comparison report can be exported as a JSON file.

## 11. Discussion

### 11.1. Cross-Domain Generalisation is Asymmetric by Model Type

The central finding of this project is that cross-dataset generalisation in crack detection is not a property of the domain shift alone but of the interaction between the domain shift and the inductive biases of each model type. The three models show markedly different generalisation profiles despite facing the same domain shift:

- The **classifier** generalises gracefully. EfficientNet-B4 retains 76.67–89.83% cracked recall across OOD domains, suggesting that semantic crack features (elongated low-contrast boundaries, branching structures) are domain-invariant at the level of global image statistics. The classifier’s task is binary and does not require spatial localisation, which reduces sensitivity to domain-specific texture priors.
- The **segmentor** generalises moderately. U-Net achieves 73.61–86.92% mIoU across the two source domains in its test split, with 13 pp of variance attributable to morphological differences (narrow irregular CrackForest cracks vs. wider higher-contrast Masonry cracks). The segmentor requires spatial reasoning but operates at pixel level, and its encoder-decoder architecture with skip connections preserves fine spatial information that supports generalisation to new crack morphologies.
- The **detector** does not generalise without adaptation. YOLOv8s collapses to near-zero mAP on heritage surfaces (4.76% and 1.13%). Object detectors encode strong spatial priors through anchor box statistics and through the feature pyramid scale assignments that determine which detection head handles which object size. OmniCrack30k crack bounding boxes have specific aspect ratio and size distributions (large pavement cracks at 640px resolution) that do not match heritage crack boxes (thin wall cracks at different apparent scale and aspect ratio).

This asymmetry has a practical implication: for heritage building monitoring applications, a high-quality classifier and segmentor can be deployed with relatively modest domain-specific data, but the detector requires targeted fine-tuning.

### 11.2. Domain Adaptation with Small Data is Viable but Limited

Both fine-tuning experiments (detector Task 5, classifier Task 1b) demonstrate that domain adaptation with approximately 250 heritage-specific annotated images produces significant and practically meaningful improvements. Detector mAP@50 improves by +169% relative on the combined heritage test set; classifier heritage recall improves by +26.9 pp. These improvements are achieved with only 50 and 20 epochs of fine-tuning respectively, confirming that the ImageNet and OmniCrack pre-training provides a strong starting point.

However, the absolute post-adaptation detector performance (23.6% mAP@50) remains far below in-distribution performance (96.7% on OmniCrack30k val). The heritage domain gap requires substantially more than 250 fine-tuning images to close fully. Collecting and annotating 1,000-5,000 heritage crack images would likely produce a practical heritage-specific detector. Self-supervised pre-training on unlabelled heritage imagery (of which there is abundant supply from photogrammetric surveys and tourism photography) is a promising direction for reducing annotation requirements.

### 11.3. The Chained Pipeline Architecture Compensates for Detector Weaknesses

The sequential classifier-detector-segmentor pipeline provides a natural compensation mechanism for the detector's high false-positive rate. The classifier, which generalises well, acts as a domain-level filter: its 99.49% accuracy on intact images means that 624 of the 780 intact test images are routed away from the detector before it can generate false alarms. This architectural decision converts a precision problem (low detector precision on heritage surfaces) into a solved problem (high classifier accuracy on the intact/cracked binary task).

This finding suggests a general design principle for heritage inspection systems: invest heavily in classification-level quality (which requires less labelled data and generalises better) and use it to gate more specialised localisation modules that may be noisier on OOD data.

### 11.4. Synthetic Degradation Validates Classifier Generalisation

The strong Pearson correlation ( $r = 0.843$ ) between the classical filter-bank degradation score and the deep learning classifier's cracked probability on synthetically degraded images provides independent validation of the classifier's generalisation. The classifier was trained on natural photographic images; the synthetic degradation pipeline applies algorithmically generated crack overlays and colour shifts. The high correlation demonstrates that the classifier responds to visual crack features rather than training-distribution-specific statistics, and that these features are preserved under the synthetic degradation operations.

This result also validates the synthetic degradation pipeline as a data augmentation strategy: augmenting the training set with synthetically degraded versions of intact images (at Stage 2 and Stage 3 severity) would introduce crack-like visual features without requiring additional manual annotation, potentially improving generalisation to early-stage real-world crack patterns.

### 11.5. Limitations

1. **Dataset scale.** The segmentation and fine-tuning datasets (358 and 250 images respectively) are small relative to industrial inspection datasets. Results may not generalise to all heritage building typologies, particularly those with highly textured or painted surfaces not represented in Masonry and CrackForest.
2. **Single-crack class.** The detection and segmentation models treat all cracks as a single class. Different crack types (shear, tension, settlement, map cracking) have different structural implications and require different conservation responses. Multi-class crack detection would require specialised annotation.
3. **2D imaging.** All models operate on 2D photographs. Crack width estimation (in millimetres) requires calibrated measurement or photogrammetric 3D reconstruction, neither of which is within scope.
4. **Temporal data.** No real temporally aligned image pairs of the same heritage building are available; the degradation analysis relies on synthetic simulation. Longitudinal monitoring requires a dedicated data collection programme.
5. **Web application validation.** The web application has been validated on the



samples in the `samples/` directory. No large-scale real-world deployment has been conducted.

## 12. Conclusions

This paper presents a comprehensive multi-task deep learning pipeline for automated structural damage assessment of cultural heritage buildings. Three computer vision models – a binary damage classifier (EfficientNet-B4), a crack detector (YOLOv8s), and a pixel-level segmentor (U-Net/ResNet34) – are trained on publicly available datasets and evaluated both in-distribution and across domains, with the cross-dataset generalisation question as the central thread.

### Primary results:

1. **Classification (EfficientNet-B4):** 99.83% accuracy, 99.56% F1, and near-perfect AUC-ROC on HistoricalCrack18-19, exceeding all state-of-the-art targets. Zero false negatives in the failure analysis: the classifier never misses actual structural damage on the 780-image test set.
2. **Detection (YOLOv8s):** 96.7% mAP@50 on OmniCrack30k validation, exceeding the 70% target. A 62.1 pp validation-to-test gap (96.7% to 34.6%) directly demonstrates cross-domain generalisation degradation. Heritage-specific fine-tuning on 250 images recovers 23.6% mAP on the combined heritage test set, a 168% relative improvement.
3. **Segmentation (U-Net/ResNet34):** 83.56% mIoU and 81.26% Dice on the combined Masonry and CrackForest test split, exceeding the 80% mIoU target. The Focal-Tversky loss combination is critical, providing approximately 5 pp mIoU improvement over standard cross-entropy.
4. **Cross-domain generalisation is asymmetric by model type:** The classifier generalises gracefully (76–90% recall on OOD domains); the segmentor generalises moderately (74–87% mIoU); the detector collapses without adaptation (below 5% mAP on heritage domains). This asymmetry is explained by the relative sensitivity of each model type’s inductive biases to the domain shift.

### Supplementary findings:

5. SAHI sliced inference underperforms plain YOLO by 9.1 pp on heritage cracks, disconfirming the hypothesis that hairline cracks are sub-pixel at 640px resolution.
6. Multi-class damage detection on DACL10k achieves 13.5% overall mAP@50, with weathering highest (22.7%) and wetspot lowest (6.6%), demonstrating practical per-class localisation on a known hard benchmark.
7. The synthetic temporal degradation pipeline achieves Pearson  $r = 0.843$  between a classical filter-bank degradation score and the deep learning classifier’s cracked probability across four simulated damage stages, validating both the simulation realism and the classifier’s domain generalisation.
8. Classifier domain adaptation on 250 heritage images recovers +26.9 pp of cracked recall on masonry surfaces with only 0.43 pp loss on in-distribution data, confirming that selective layer fine-tuning is an efficient and effective adaptation strategy.

**Future directions** include: (1) instance segmentation with Mask R-CNN or YOLOv8-seg to unify detection and segmentation into a single model; (2) self-supervised pre-training on unlabelled heritage facade imagery to reduce the domain gap without additional annotation; (3) photogrammetric 3D integration for millimetre-level crack width estimation; (4) collection of a real longitudinal dataset through repeated photographic surveys of the same heritage structures to enable genuine temporal monitoring.

## A. Training Hyperparameter Summary

Table 20: Complete training hyperparameters for all three primary models.

| Hyperparameter      | Classifier (Task 1) | Detector (Task 2)  | Segmentor (Task 3) |
|---------------------|---------------------|--------------------|--------------------|
| Architecture        | EfficientNet-B4     | YOLOv8s            | U-Net / ResNet34   |
| Parameters          | 19M                 | 11.2M              | 24M                |
| Input resolution    | 380×380             | 416×416            | 384×384            |
| Optimiser           | AdamW               | SGD (built-in)     | AdamW              |
| Initial LR          | $1 \times 10^{-4}$  | 0.01               | $3 \times 10^{-4}$ |
| LR schedule         | Cosine annealing    | Cosine decay       | Cosine annealing   |
| Weight decay        | $10^{-2}$           | $5 \times 10^{-4}$ | $10^{-4}$          |
| Batch size          | 16 (eff. 32)        | 32                 | 16                 |
| Epochs              | 30 (early stop)     | 30                 | 150 (early stop)   |
| Early stop patience | 5 on val F1         | Not used           | 15 on val mIoU     |
| Loss function       | Weighted CE         | YOLOv8 default     | Focal + Tversky    |
| AMP                 | Yes                 | Yes                | Yes                |
| Hardware            | T4 GPU (Colab)      | T4 GPU (Colab)     | T4 GPU (Colab)     |
| Training time       | ≈1.5 hr             | ≈3.6 hr            | ≈4.0 hr            |

## B. Software Environment

Table 21: Key software versions used for training and evaluation.

| Package                     | Version |
|-----------------------------|---------|
| Python                      | 3.10    |
| PyTorch                     | 2.1.0   |
| torchvision                 | 0.16.0  |
| ultralytics (YOLOv8)        | 8.0.196 |
| segmentation-models-pytorch | 0.3.3   |
| timm (EfficientNet)         | 0.9.7   |
| pytorch-grad-cam            | 1.4.8   |
| OpenCV                      | 4.8.1   |
| scikit-image                | 0.22.0  |
| FastAPI                     | 0.104.1 |
| SAHI                        | 0.11.14 |
| CUDA                        | 12.2    |

## References

- [1] Ronneberger, O., Fischer, P., and Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241.
- [2] Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., and Adam, H. (2018). Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 801–818.
- [3] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(6), pp. 1137–1149.
- [4] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- [5] Tan, M. and Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the International Conference on Machine Learning (ICML)*, pp. 6105–6114.
- [6] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2020). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *International Journal of Computer Vision*, 128(2), pp. 336–359.
- [7] Dais, D., Bal, I. E., Smyrou, E., and Sarhosis, V. (2021). Automatic crack classification and segmentation on masonry surfaces using convolutional neural networks and transfer learning. *Automation in Construction*, 125, 103606.
- [8] Mishra, M. and Loureno, P. B. (2024). Deep learning and computer vision for damage detection in cultural heritage structures: A review. *Journal of Cultural Heritage*, 65, pp. 275–292.
- [9] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788.
- [10] Jocher, G., Chaurasia, A., and Qiu, J. (2023). Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>
- [11] Benz, C. and Rodehorst, V. (2022). OmniCrack30k: A benchmark for crack segmentation and the multi-task learning of cracks. *arXiv preprint arXiv:2203.07033*.
- [12] Benz, C., Flotzinger, J., and Rodehorst, V. (2023). DACL10k: Benchmark for semantic bridge damage segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*.
- [13] Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollar, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988.